

Topic 8

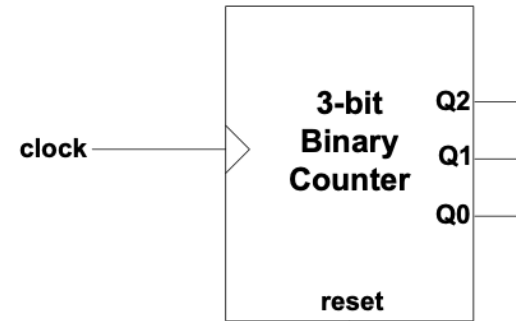
Counters

Counters

- A digital circuit that counts in different format: binary, decimal, signed, unsigned...
- An n -bit binary counter consists of n flip-flops
- An n -bit binary counter can count at most 2^n-1 integers
- Can count up or down: increment or decrement once per clock cycle
- Counters:
 - Asynchronous counters
 - Synchronous counters

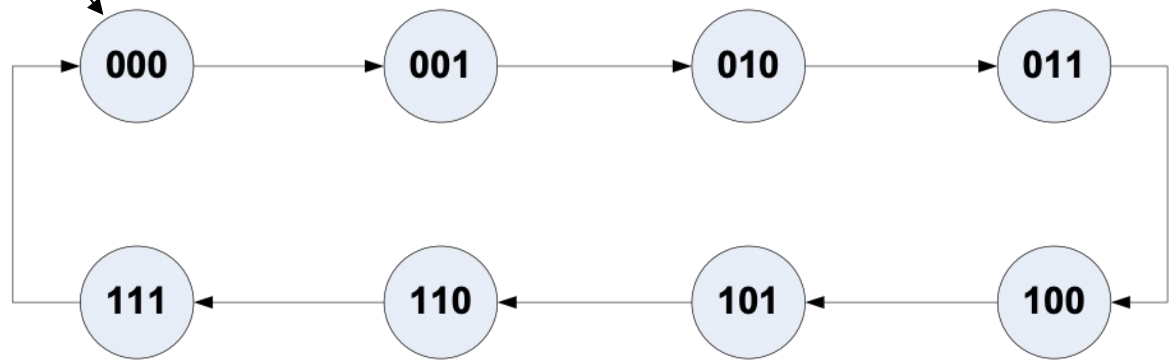
Synchronous Binary Counter

- **Synchronous counter:** all the flip-flops are triggered by (synchronized to) the same clock
- May be implemented by different type of flip-flops (D, T, or JK)



- Example: a 3-bit binary counter can count through this sequence

Start (reset)



Synchronous Binary Counter Design

- Following table describes the behavior of the counter

Current Value (state)			Next Value (state)		
Q2	Q1	Q0	Q2 ⁺	Q1 ⁺	Q0 ⁺
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	0

Counter Implemented with D Flip-Flop

- Use D flip flops to hold values: **$Q^+ = D$ upon active edge**

Present State			Next State			D flip flop input		
Q2	Q1	Q0	Q2 ⁺	Q1 ⁺	Q0 ⁺	D2	D1	D0
0	0	0	0	0	1	0	0	1
0	0	1	0	1	0	0	1	0
0	1	0	0	1	1	0	1	1
0	1	1	1	0	0	1	0	0
1	0	0	1	0	1	1	0	1
1	0	1	1	1	0	1	1	0
1	1	0	1	1	1	1	1	1
1	1	1	0	0	0	0	0	0

The diagram shows two blue curly braces. The first brace is under the columns Q2⁺, Q1⁺, and Q0⁺. The second brace is under the columns D2, D1, and D0. A blue arrow points from the word "Same" to the space between these two braces, indicating that the next state values are equal to the D flip flop inputs.

Same

Counter Implemented with D Flip-Flop

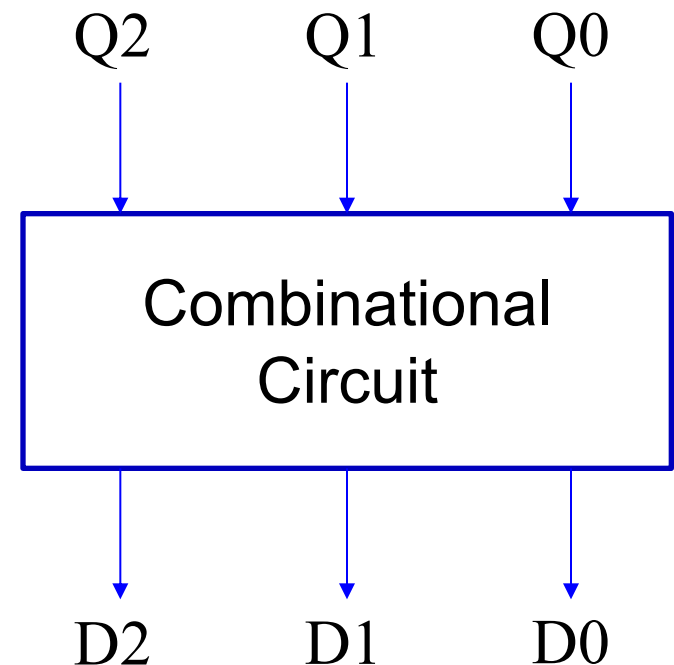
- Drop the columns for Q^+

Present State (D-FF outputs)			Next State (D-FF inputs)		
Q2	Q1	Q0	D2	D1	D0
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	0

Truth table inputs Truth table outputs



From outputs of DFFs



To inputs of DFFs

Counter Implemented with D Flip-Flop

Karnaugh map for D2:

		Q1Q0			
		00	01	11	10
Q2	0	0	0	1	0
	1	1	1	0	1

$$\begin{aligned}
 D2 &= Q2Q1' + Q2Q0' + Q2'Q1Q0 \\
 &= Q2(Q1' + Q0') + Q2'Q1Q0 \\
 &= Q2(Q1Q0)' + Q2'(Q1Q0) \\
 &= Q2 \oplus (Q1Q0)
 \end{aligned}$$

Karnaugh map for D1:

		Q1Q0			
		00	01	11	10
Q2	0	0	1	0	1
	1	0	1	0	1

$$\begin{aligned}
 D1 &= Q1'Q0 + Q1Q0' \\
 &= Q1 \oplus Q0
 \end{aligned}$$

Karnaugh map for D0:

		Q1Q0			
		00	01	11	10
Q2	0	1	0	0	1
	1	1	0	0	1

$$D0 = Q0'$$

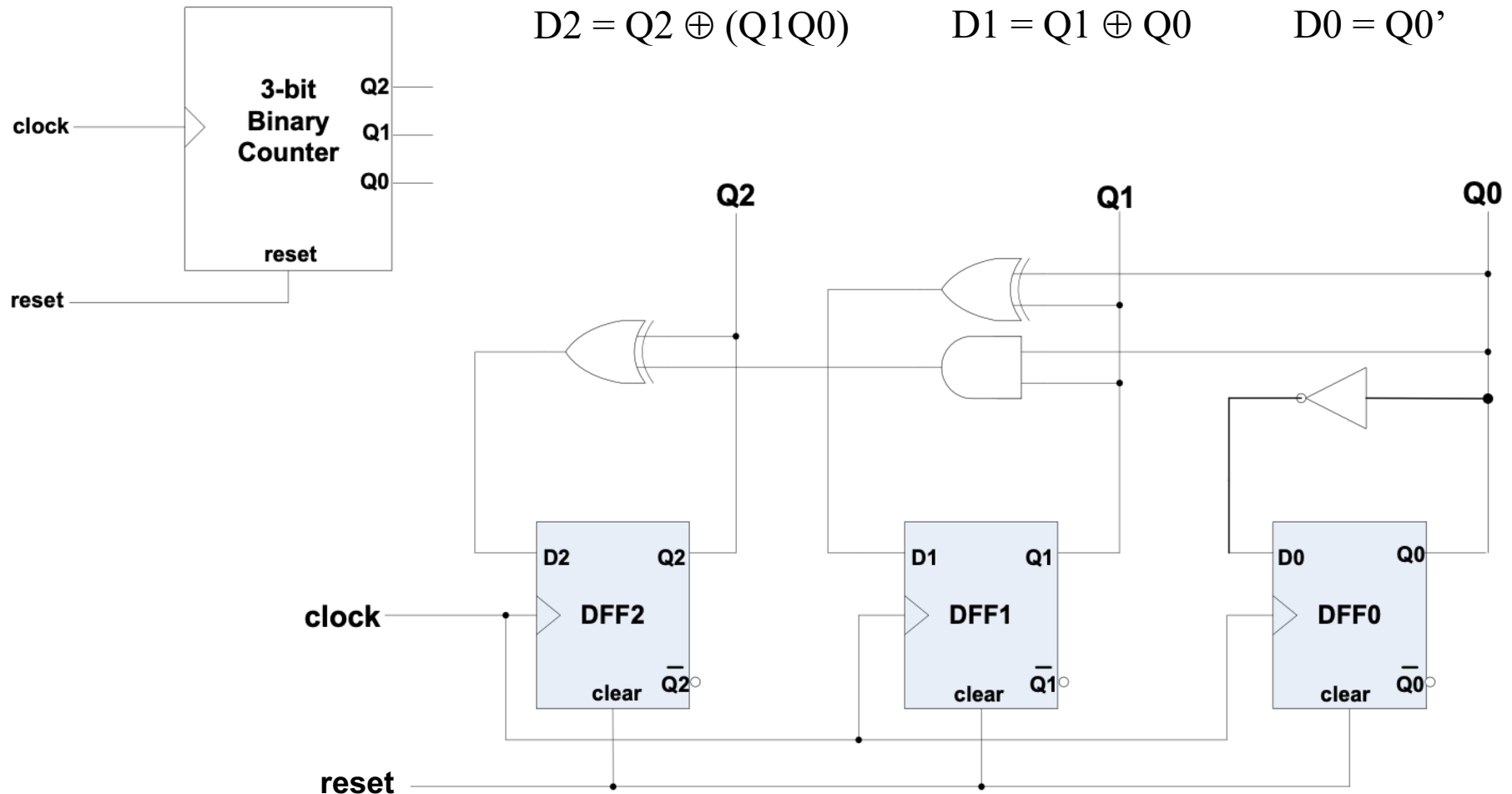
- The input equation can be generalized as

$$Dn = Qn \oplus (Qn-1 \dots Q1Q0)$$

Present State			D flip flop input		
Q2	Q1	Q0	D2	D1	D0
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	1	1	0
1	1	0	1	1	1
1	1	1	0	0	0

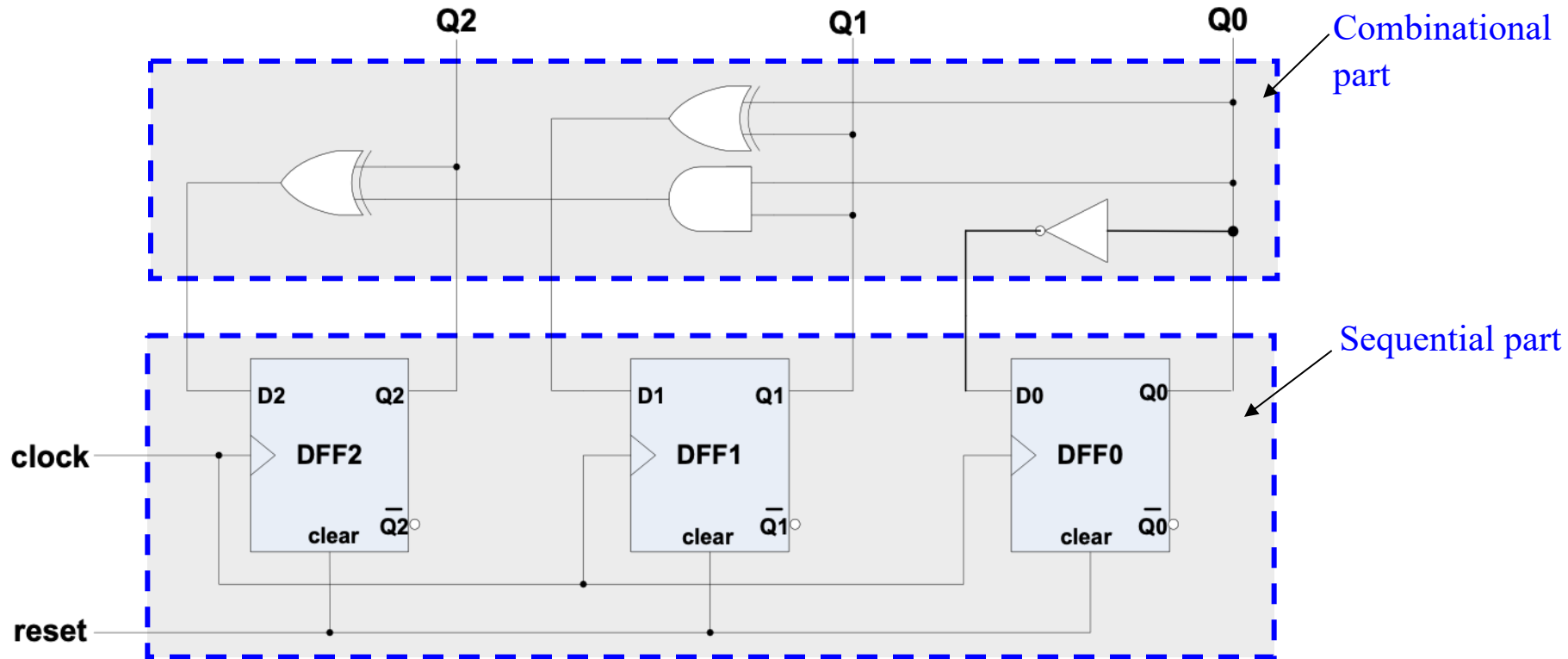
Synchronous Binary Counter with D Flip-Flop

- 3-bit binary counter by D FF with asynchronous clear/reset



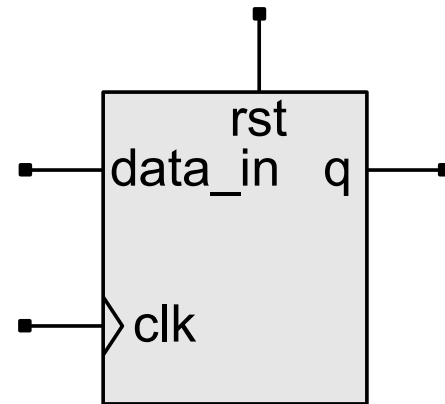
Synchronous Binary Counter with D Flip-Flop

- 3-bit binary counter by D FF



DFF with Asynchronous Reset

```
module D_ff (q, data_in, clk, rst);  
  input data_in, clk, rst;  
  output q;  
  
  reg q;  
  
  always @ (posedge clk or posedge rst)  
  begin  
    if (rst == 1) q <= 0;  
    else q <= data_in;  
  end  
endmodule
```



Verilog Module: Synchronous Binary Counter

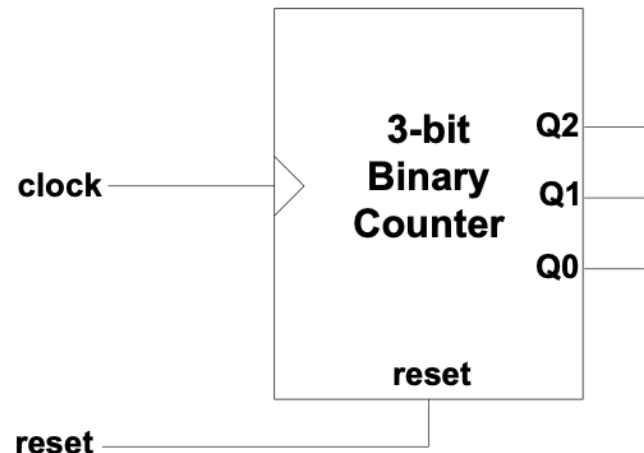
```
module counter_3_bit (clock, reset, Q);  
  input      clock, reset;  
  output     [2:0] Q;  
  wire       D2, D1, D0, and_out;  
  
  D_ff DFF2 (Q[2], D2, clock, reset);  
  D_ff DFF1 (Q[1], D1, clock, reset);  
  D_ff DFF0 (Q[0], D0, clock, reset);  
  xor (D2, Q[2], and_out);  
  xor (D1, Q[1], Q[0]);  
  and (and_out, Q[1], Q[0]);  
  not (D0, Q[0]);  
endmodule
```

Sequential part

Combinational part

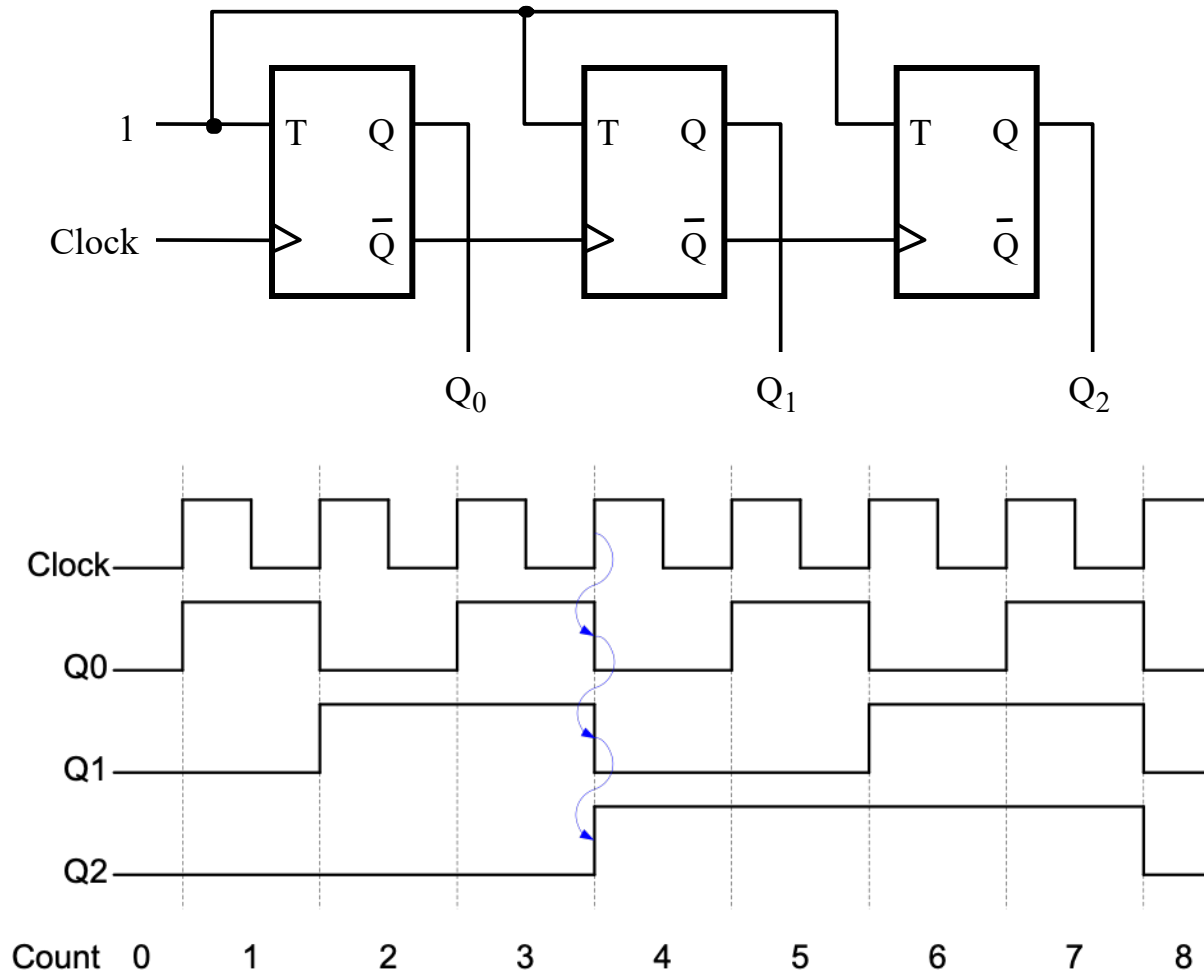
Verilog Model: Synchronous Binary Counter

```
module counter_N_bit (clock, reset, Q);  
    parameter N = 3; ← Defines a constant N  
    input      clock, reset;  
    output     [N-1:0]    Q;  
    reg        [N-1:0]    Q;  
    always @ (posedge reset or posedge clock)  
        if (reset == 1'b1) Q <= 0;  
        else                Q <= Q + 1; //behavioral style  
endmodule
```



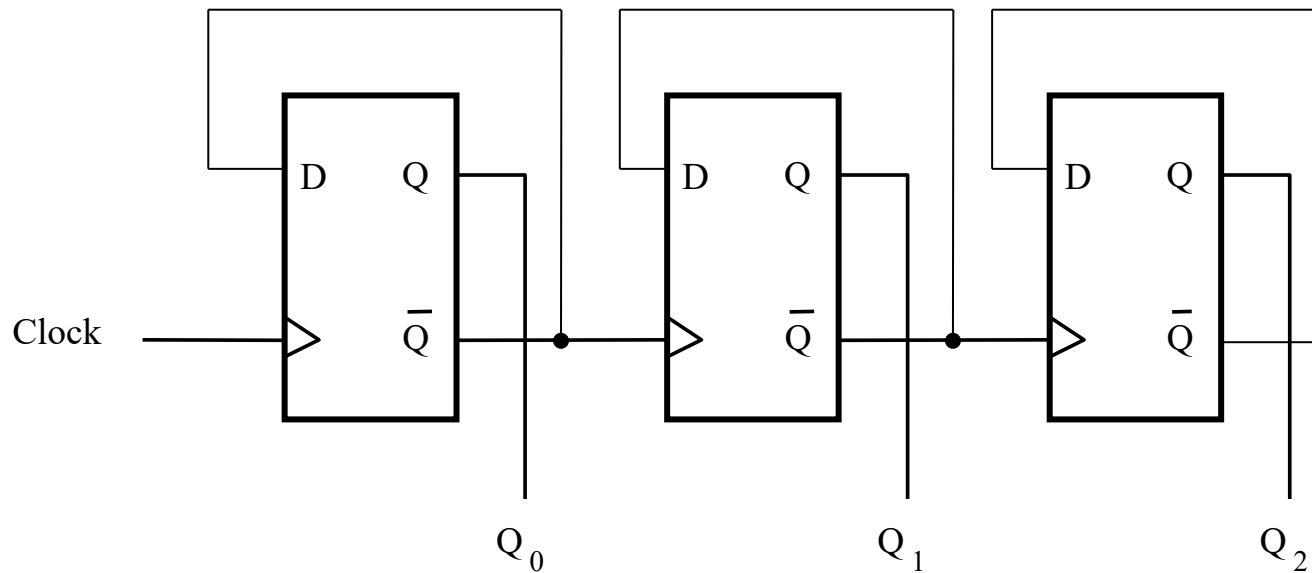
Asynchronous Binary Counter – Ripple Counter

- Flip flops are not triggered by a global clock signal
 - Example: up counter with T flip flops



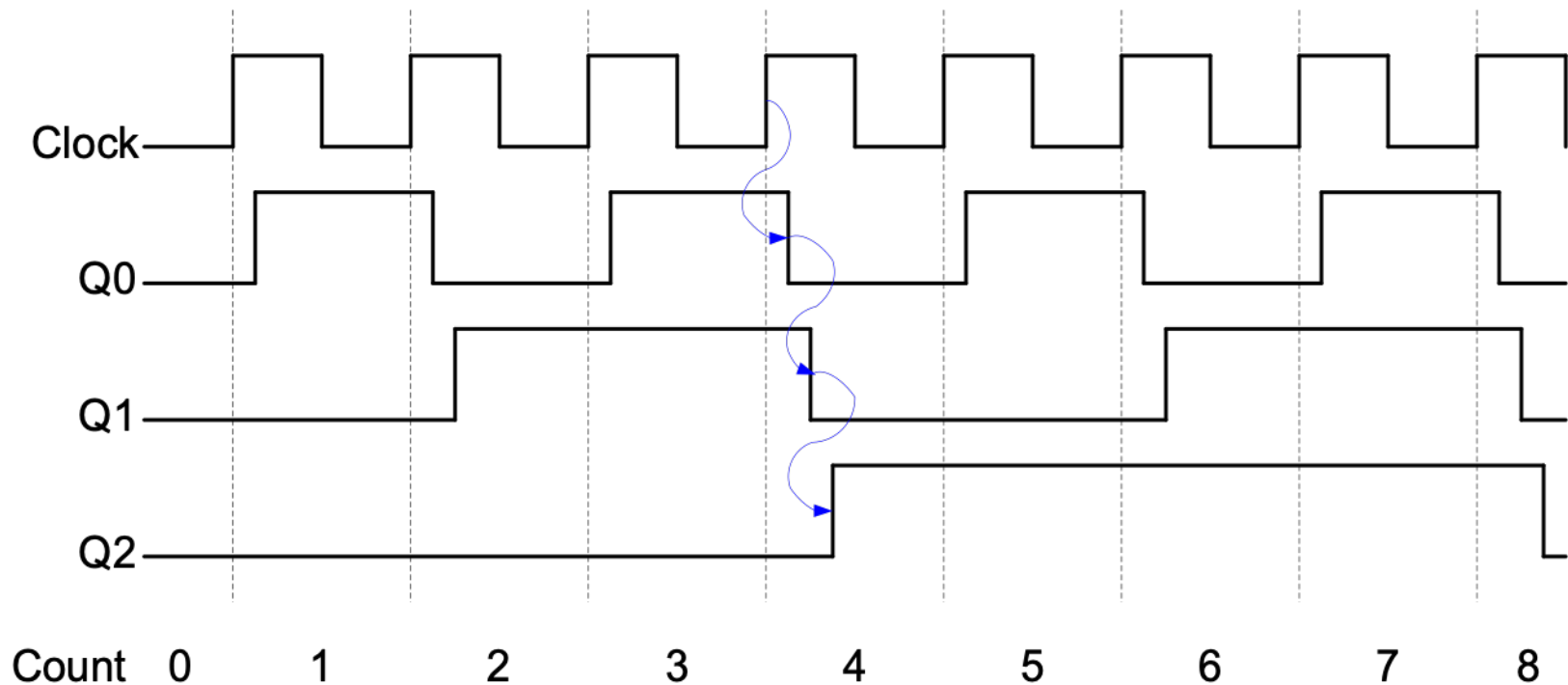
Asynchronous Binary Counter – Ripple Counter

- Example:
 - Alternative binary ripple counter, with D flip flops, up counter



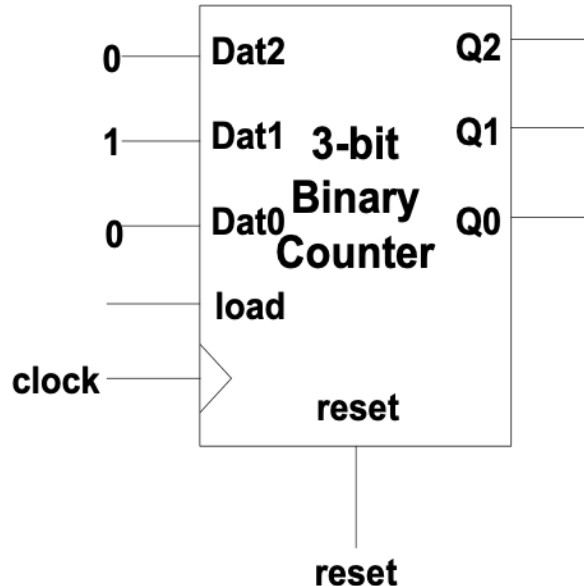
Asynchronous Binary Counter – Ripple Counter

- Problem with asynchronous counters:
 - Delays caused by each stage – timing issues



Synchronous Counter with Load

- Control inputs may be added to the flip flops in a binary counter to control the behavior of the counter
- Load (Parallel Load):**
 - Numbers can be loaded into the counter when the load input is high, thus the count sequence can be customized

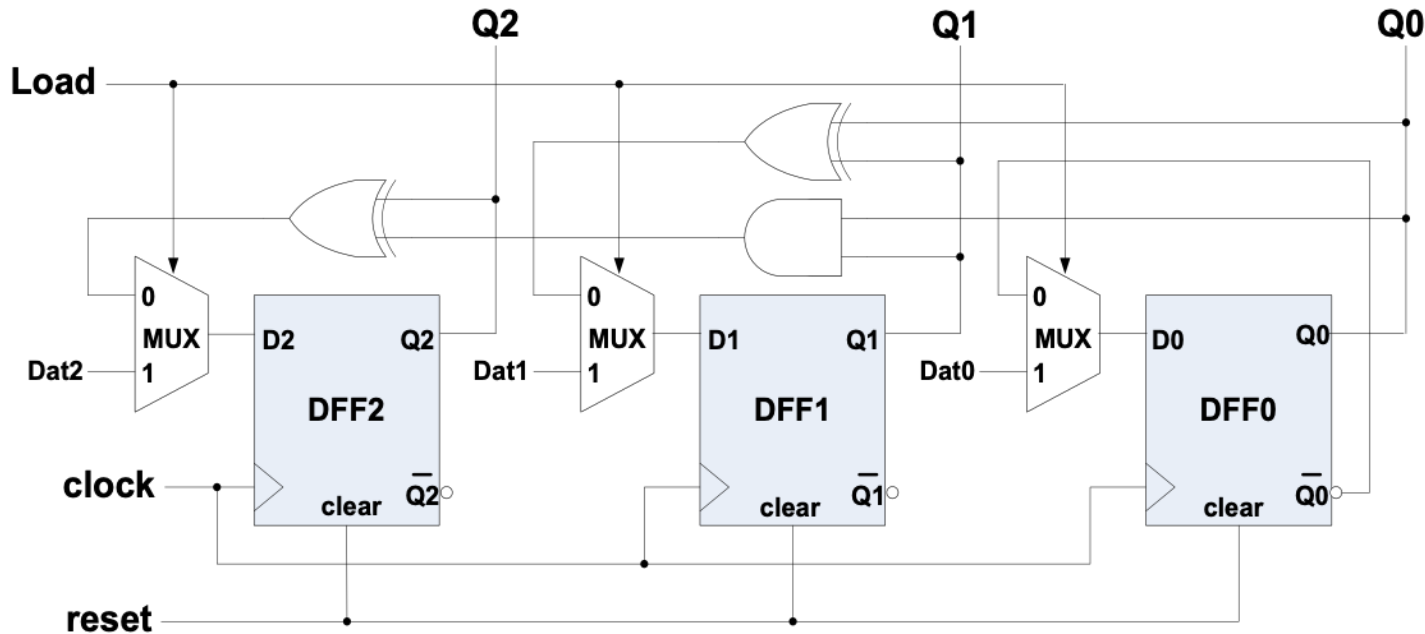


Counting sequence:

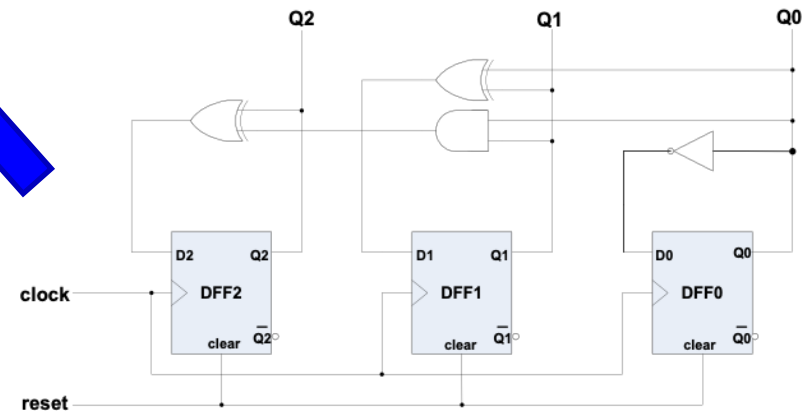
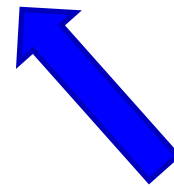
010 → ... → 110 → 111 → 000 → 001 ...

Implementation of Parallel Load

- Using MUXes for the D inputs of DFFs



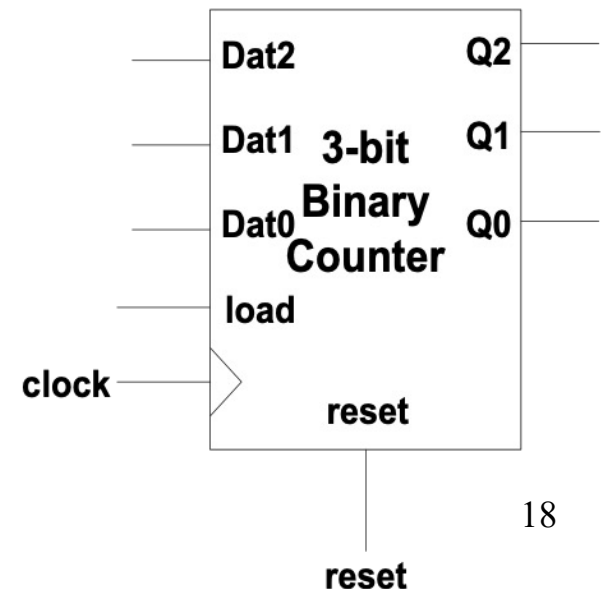
Note: Load is a
synchronous control signal



Verilog Model: Counter with Parallel Load

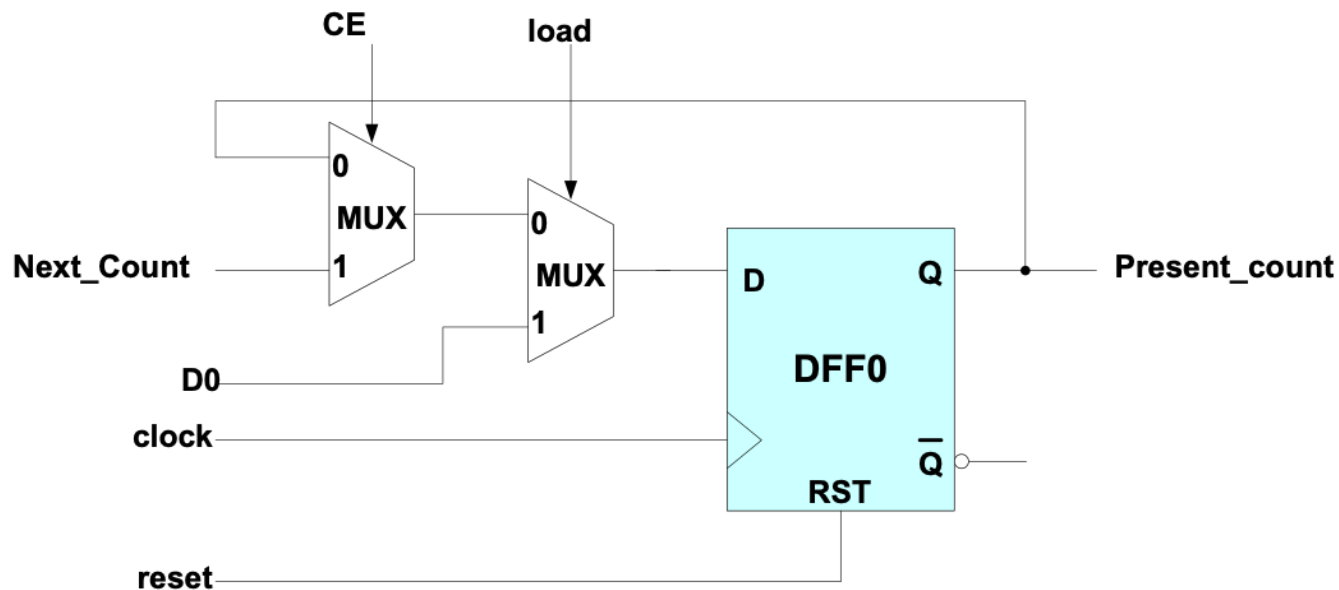
```
module counter_N_bit (clock, reset, load, Dat, Q);  
    parameter N = 3;  
    input      clock, reset, load;  
    input      [N-1:0]    Dat;  
    output     [N-1:0]    Q;  
    reg        [N-1:0]    Q;  
    always @ (posedge reset or posedge clock)  
        if (reset == 1'b1) Q <= 0;  
        else if (load == 1'b1) Q <= Dat;  
        else Q <= Q + 1;  
endmodule
```

Synchronous Control



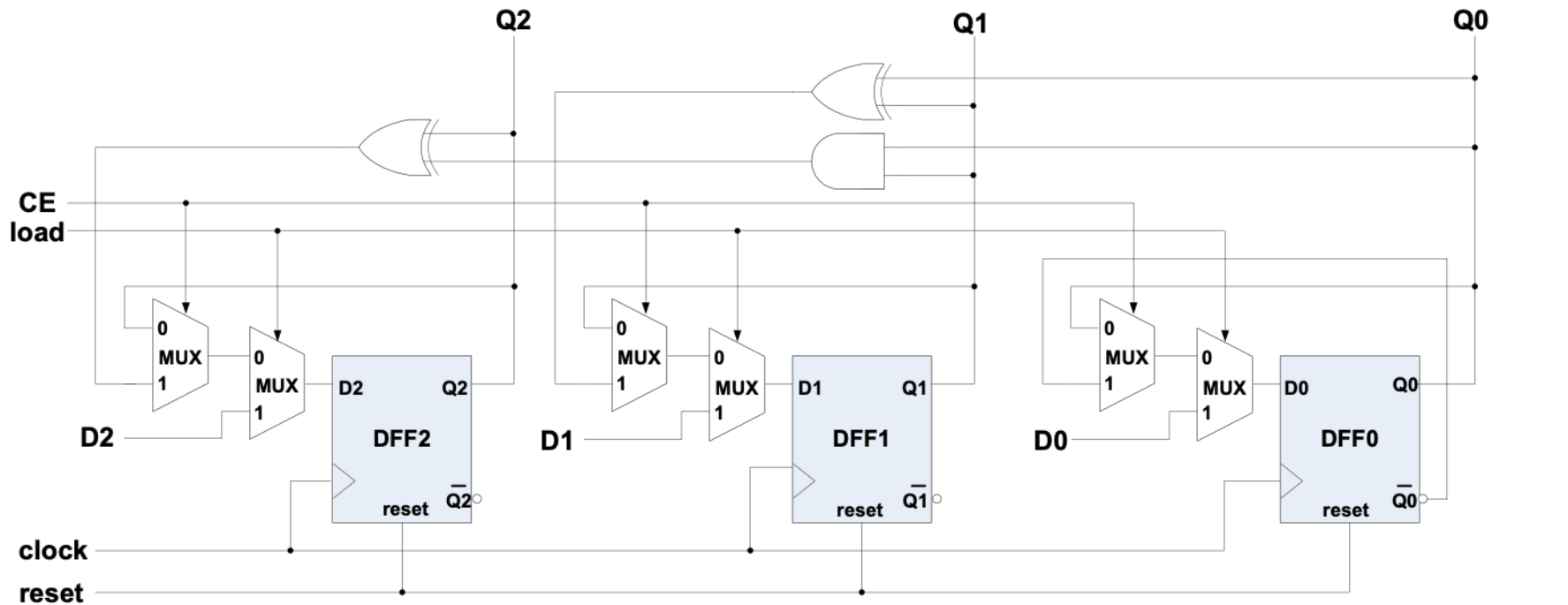
Synchronous Counter with Count Enable

- **Count Enable (CE) :**
 - when $CE = 1$, counter counts
 - when $CE = 0$, counter holds the values
- Used to hold the counter to
 - Wait certain acknowledge signal coming from other devices
 - Concatenate small counters into bigger ones
- Implementation of 1-bit of a counter with both CE and load:



Implementation of CE and Load

- 3-bit binary counter with load and CE



Priority of External Control Signals

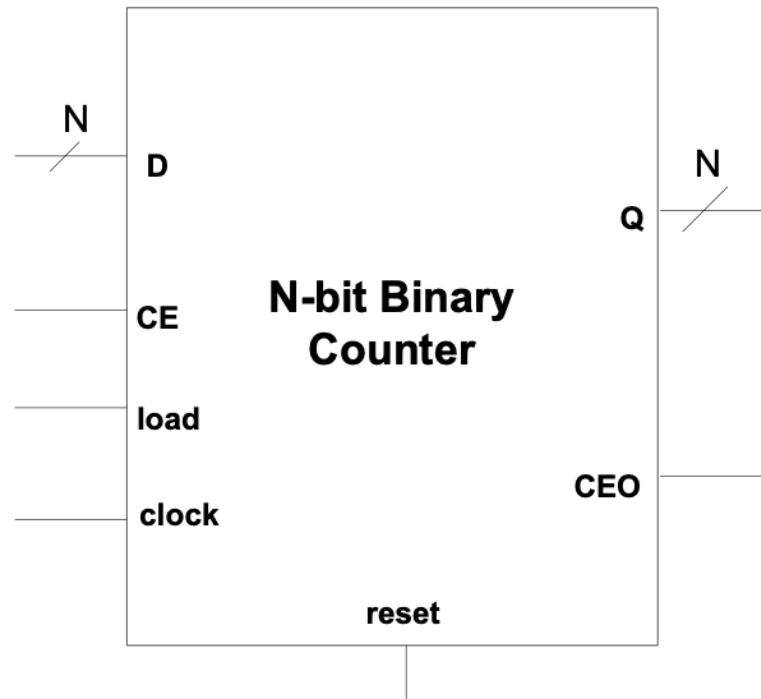
- Function table of a counter with external controls
 - a prioritized hierarchical control structure

reset	load	CE	Action on the rising clock edge
1	X	X	Clear ($Q_n \leq 0$)
0	1	X	Load ($Q_n \leq D_n$)
0	0	1	Count
0	0	0	Hold (No Change)

N-bit Binary Counter

- Count Enable Output (CEO)

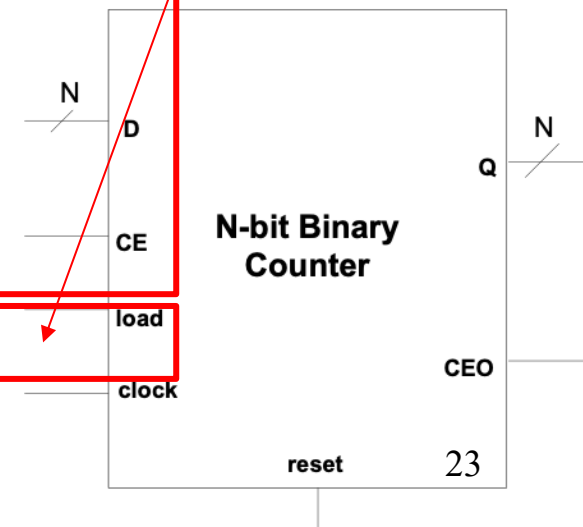
$$\text{CEO} = \text{CE} \cdot Q_{N-1} \cdot Q_{N-2} \cdot \dots \cdot Q_0$$



Verilog Model: Counter with Count Enable

```
module counter_N_bit (clock, reset, load, CE, D, Q, CEO);  
    parameter N = 3;  
    input      clock, reset, load, CE;  
    input      [N-1:0] D;  
    output     [N-1:0] Q;  
    output     CEO;  
    reg        [N-1:0] Q;  
  
    always @ (posedge reset or posedge clock)  
        if (reset == 1'b1) Q <= 0;  
        else if (load == 1'b1) Q <= D;  
        else if (CE == 1'b1) Q <= Q + 1;  
        else Q <= Q;  
  
    assign CEO = CE & (&Q);  
endmodule
```

Executed
concurrently



Testbench Written in Verilog

```
module Test_Banch;
  parameter half_period = 50;
  parameter counter_size = 4;

  wire [counter_size-1:0] Q;
  reg [counter_size-1:0] Din;
  reg clock, load, reset, CE;
```

```
  counter_N_bit #(counter_size) UUT (clock,reset,load,CE,Din,Q,CEO);
```

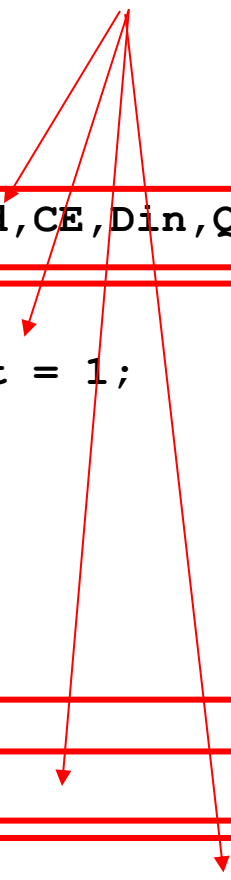
```
  initial begin
    #0    clock = 0; Din = 0; load = 0; CE = 1; reset = 1;
    #100  reset = 0;
    #200  Din = 8; load = 1;
    #100  load = 0;
    #300  CE = 0;
    #200  CE = 1;
  end
```

```
  always #half_period clock = ~clock;
```

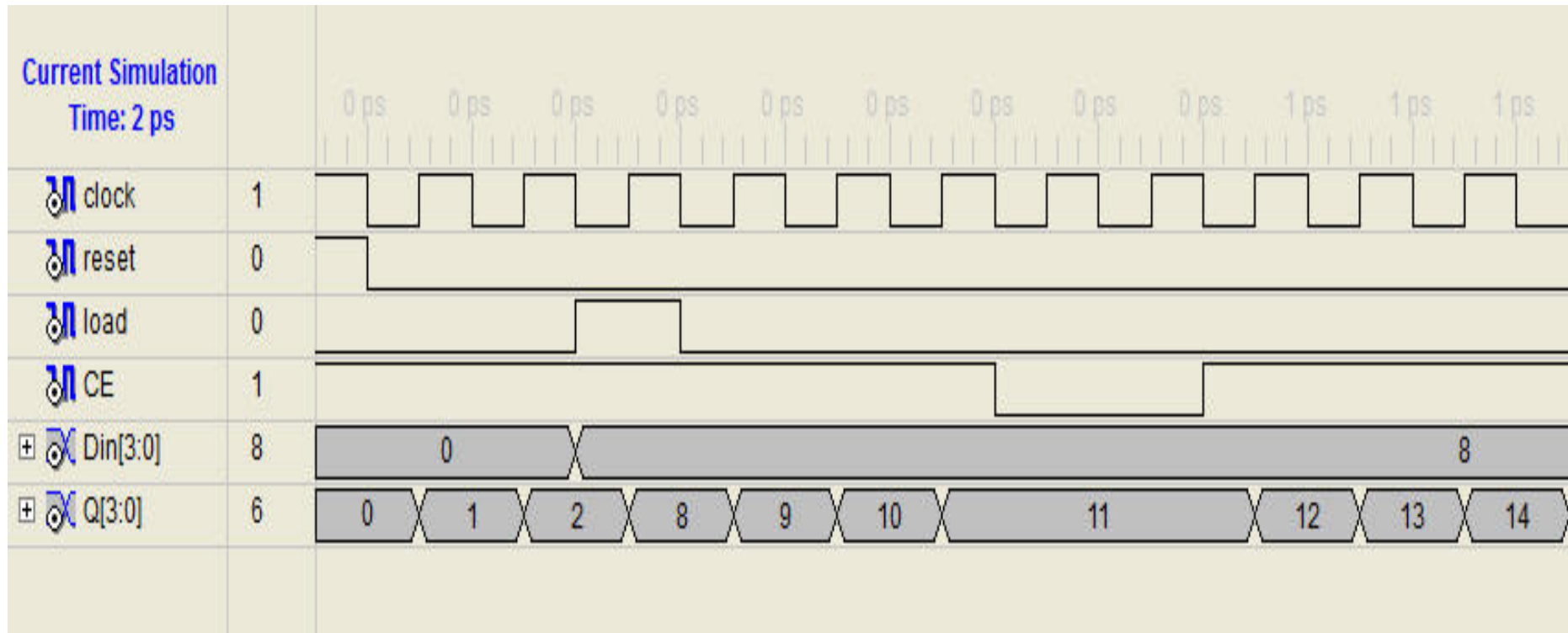
```
  initial #2000 $stop;
```

```
endmodule
```

All executed
concurrently

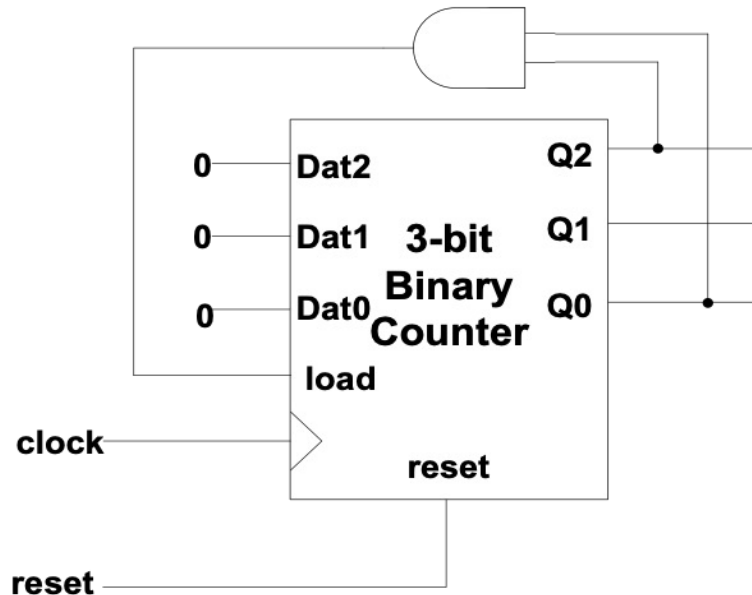


Simulation Result



Customize Counting Sequence

- What does this circuit do?

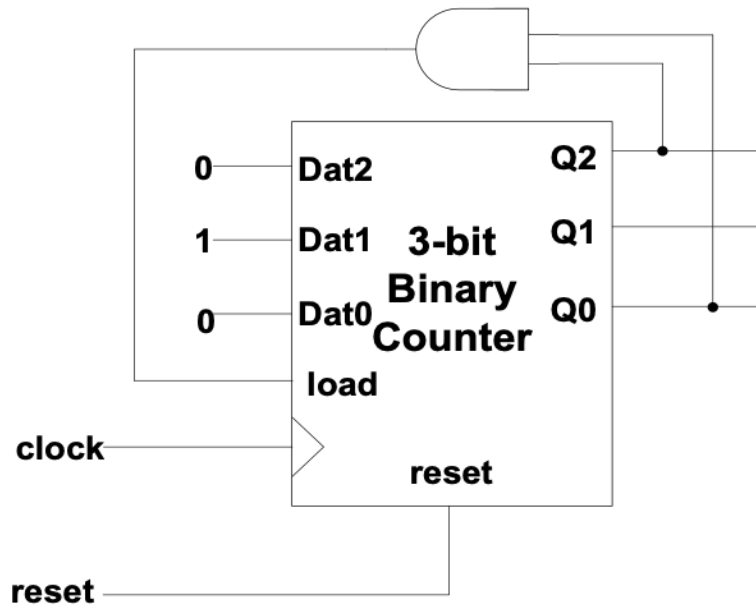


Q2	Q1	Q0	Q2 ⁺	Q1 ⁺	Q0 ⁺
0	0	0	0	0	1
0	0	1	0	1	0
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	0	0	0
1	1	0	X		
1	1	1			

- load = Q2Q0, initial number = 000
- The count sequence now becomes
000 → 001 → ... → 101 → 000
(modulo-6 counter)

Customize Counting Sequence

- What does this do?



Q2	Q1	Q0	Q2 ⁺	Q1 ⁺	Q0 ⁺
0	0	0	X		
0	0	1			
0	1	0	0	1	1
0	1	1	1	0	0
1	0	0	1	0	1
1	0	1	0	1	0
1	1	0	X		
1	1	1			

- load = Q2Q0, initial number = 010
- The count sequence is
 $000 \rightarrow \dots \rightarrow 101 \rightarrow 010 \rightarrow 011 \rightarrow \dots \rightarrow 101 \rightarrow 010 \rightarrow \dots$

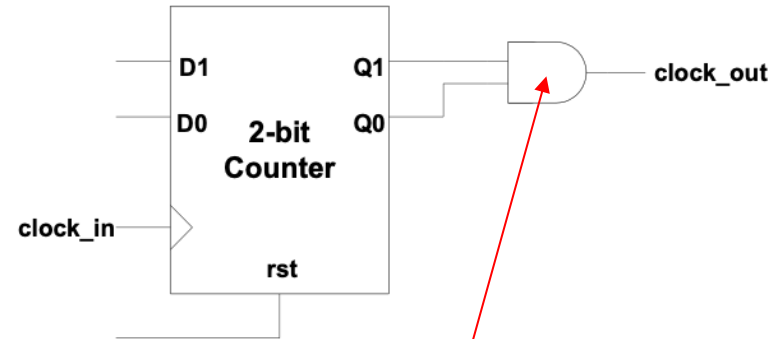
In first cycle

An Application of Counter: Clock Divider

- Slows a clock down by reducing its frequency
 - Generates clock signal with bigger clock cycle
 - Input clock can be slowed down by odd or even number of times
 - For slower devices and lower power consumption

Example: Divide by 4

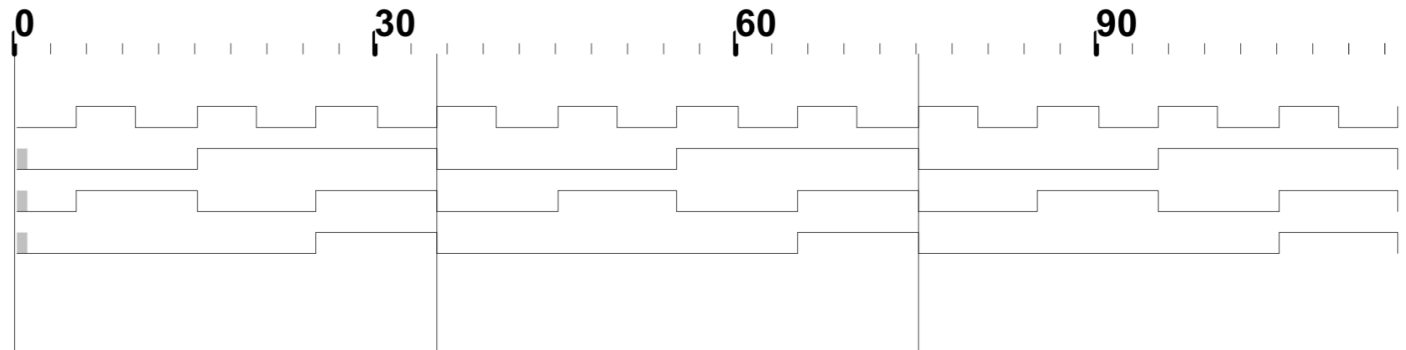
- 4-fold clock divider: 2-bit binary counter
- The clock_out generates one pulse for every four input clock cycles.
 - So the frequency of the clock_in is reduced by 4 times
- Q1 serves the same purpose with 50% duty cycle



Ideal case, delay ignored

T1 35 T2 75 Tdelta 40

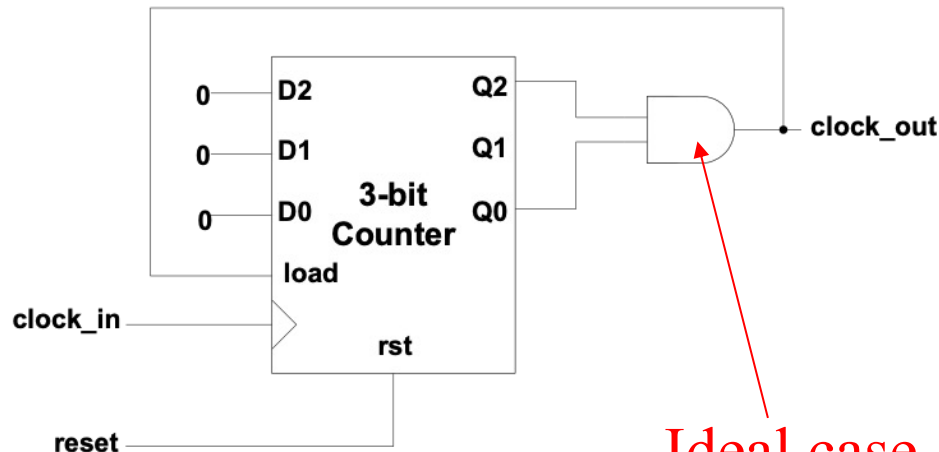
Name
clock_divider
clock_in
Q1
Q0
clock_out



Both have the same slower frequency

Example: Divide by 6

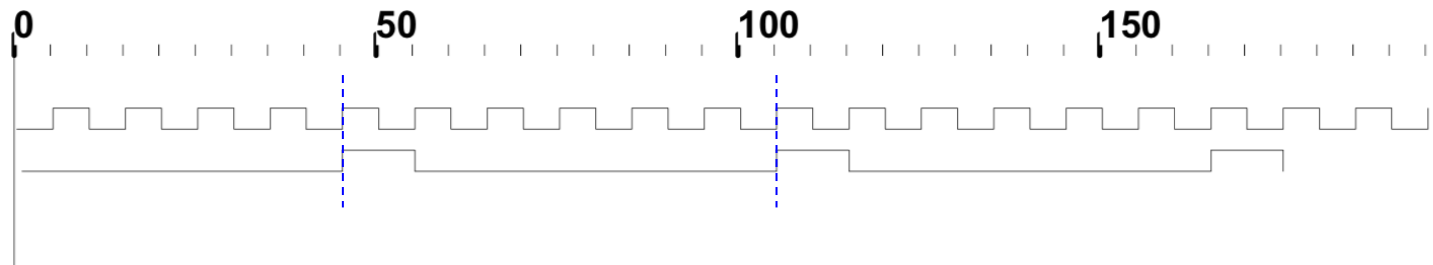
- $2^2 < 6 < 2^3$, at least a 3-bit counter
- Counting sequence: $000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow 100 \rightarrow 101 \rightarrow 000$
- The output should be “1” whenever $Q_2Q_1Q_0$ are 111
 $\text{clock_out} = \text{load} = Q_2 \& Q_1' \& Q_0 = Q_2 \& Q_0$



Ideal case, delay ignored

T1 T2 Tdelta

Name
clock_divider
clock_in
clock_out

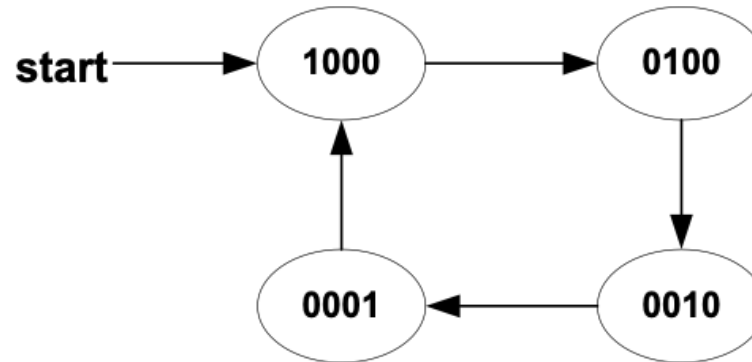


Divide by N

- In general, for N-fold Clock Divider
 - $2^{n-1} < N \leq 2^n$, where n is the size of counter
 - $N-1$ should be the upper bound of the counter's counting sequence, i.e.
 $0 \rightarrow 1 \rightarrow \dots \rightarrow N-1 \rightarrow 0$
 - Thus the counter should be force back to 0 using load input when it reaches the number $N-1$
 - If B_{N-1} is the binary equivalent of decimal number $N-1$, both load and output clock can be formed by ANDing the bits of '1's in B_{N-1}

Ring Counter

- A ring counter is a circular shifter with only one FF being set at any time



Current				Next			
1	0	0	0	0	1	0	0
0	1	0	0	0	0	1	0
0	0	1	0	0	0	0	1
0	0	0	1	1	0	0	0
all other inputs				X			

D3 = Q0

D2 = Q3

D1 = Q2

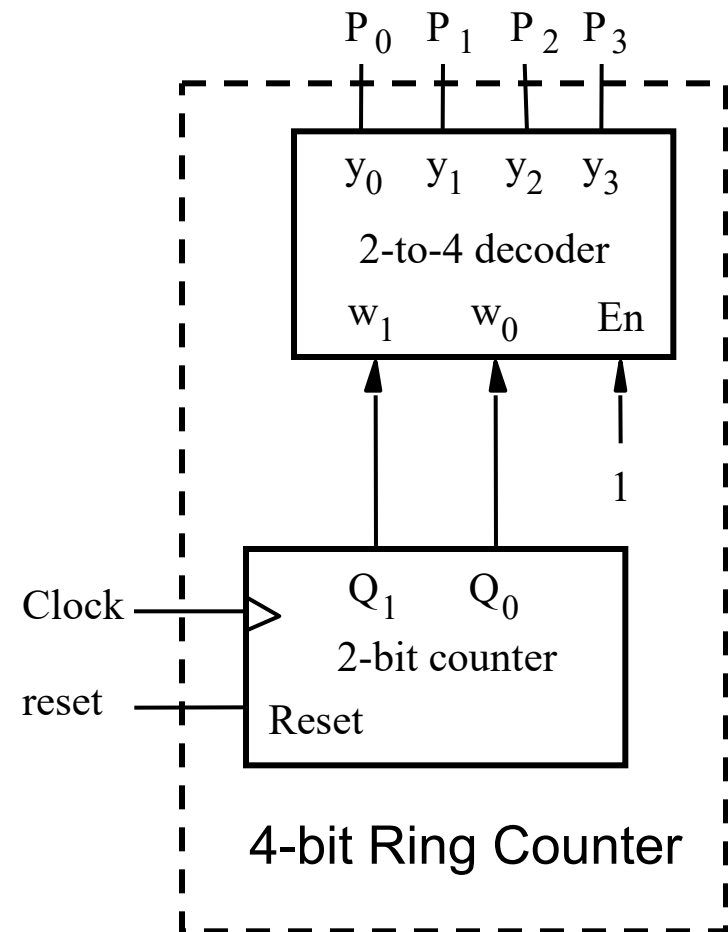
D0 = Q1

Ring Counter – Design Alternative

- A ring counter is a circular shift register with only one FF being set at any time

w_1	w_0	P_0	P_1	P_2	P_3
0	0	1	0	0	0
0	1	0	1	0	0
1	0	0	0	1	0
1	1	0	0	0	1

functional table of 2×4 Decoder



Johnson Counter

- Counting sequence

1000 → 1100 → 1110 → 1111 → 0111 → 0011 → 0001 → 0000 → ... repeat

- In each state transition, only one bit changes
 - less state transition effort
 - but less counting state too

Q3	Q2	Q1	Q0	Q3 ⁺	Q2 ⁺	Q1 ⁺	Q0 ⁺
0	0	0	0	1	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	x	x	x	x
0	0	1	1	0	0	0	1
0	1	0	0	x	x	x	x
0	1	0	1	x	x	x	x
0	1	1	0	x	x	x	x
0	1	1	1	0	0	1	1
1	0	0	0	1	1	0	0
1	0	0	1	x	x	x	x
1	0	1	0	x	x	x	x
1	0	1	1	x	x	x	x
1	1	0	0	1	1	1	0
1	1	0	1	x	x	x	x
1	1	1	0	1	1	1	1
1	1	1	1	0	1	1	1

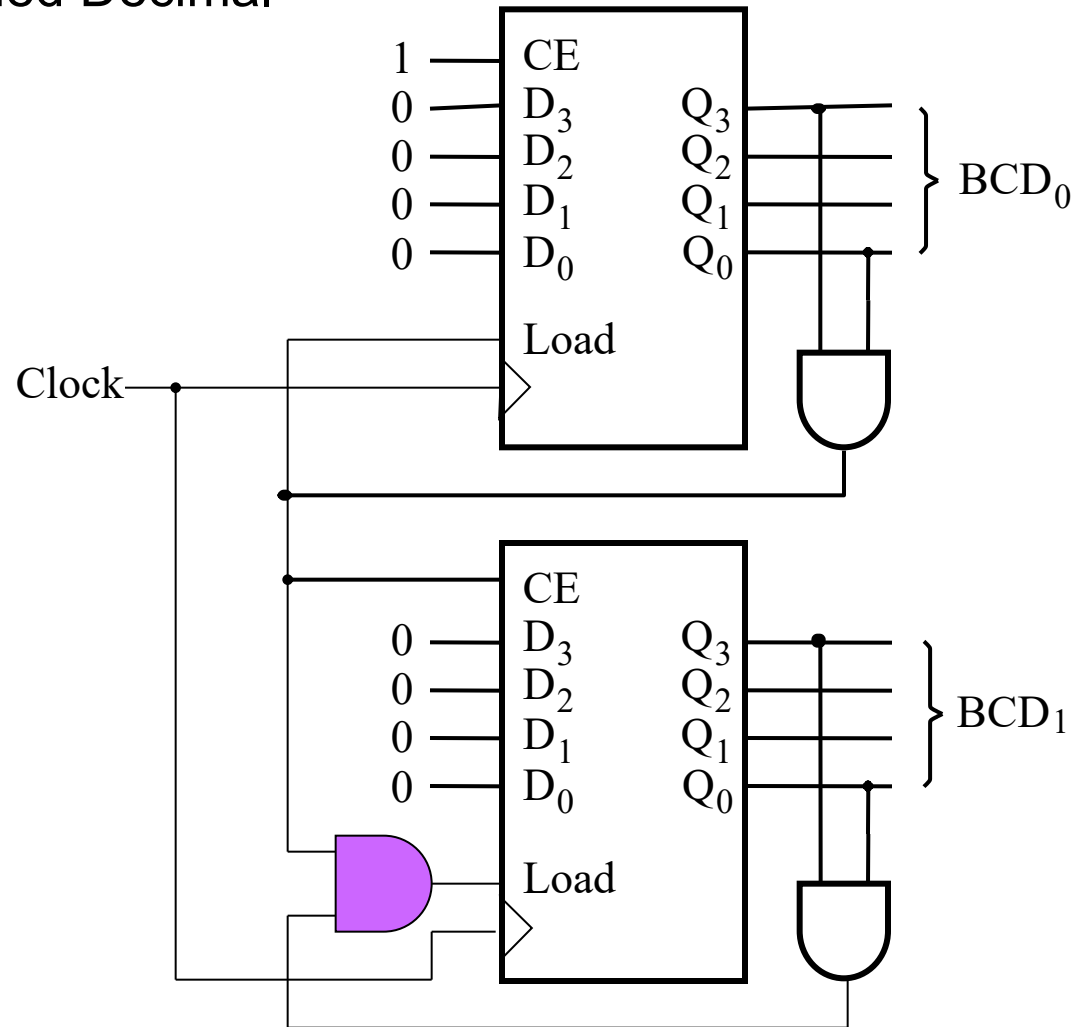
BCD Counter

- A counter doesn't have to follow through the entire counting sequence
- Example: BCD (Binary Coded Decimal) counter

Q3	Q2	Q1	Q0	Q3 ⁺	Q2 ⁺	Q1 ⁺	Q0 ⁺
0	0	0	0	0	0	0	1
0	0	0	1	0	0	1	0
0	0	1	0	0	0	1	1
0	0	1	1	0	1	0	0
0	1	0	0	0	1	0	1
0	1	0	1	0	1	1	0
0	1	1	0	0	1	1	1
0	1	1	1	1	0	0	0
1	0	0	0	1	0	0	1
1	0	0	1	0	0	0	0
1	0	1	0	X			
		.					
		.					
		.		X			
1	1	1	1				

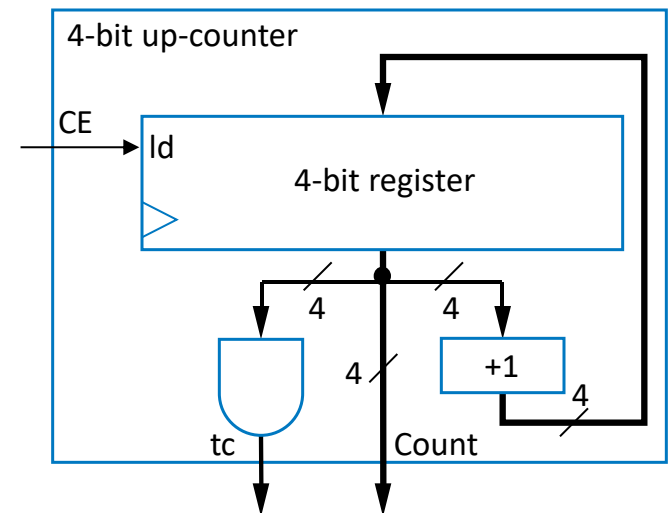
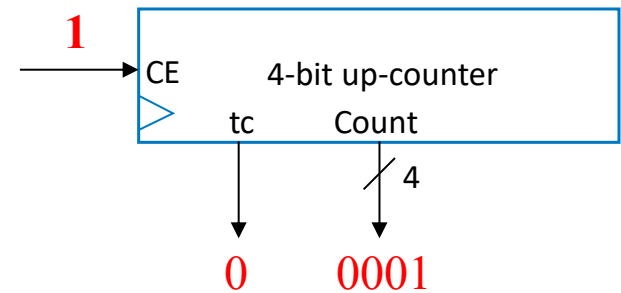
2-Digit BCD Counter

- BCD: Binary Coded Decimal



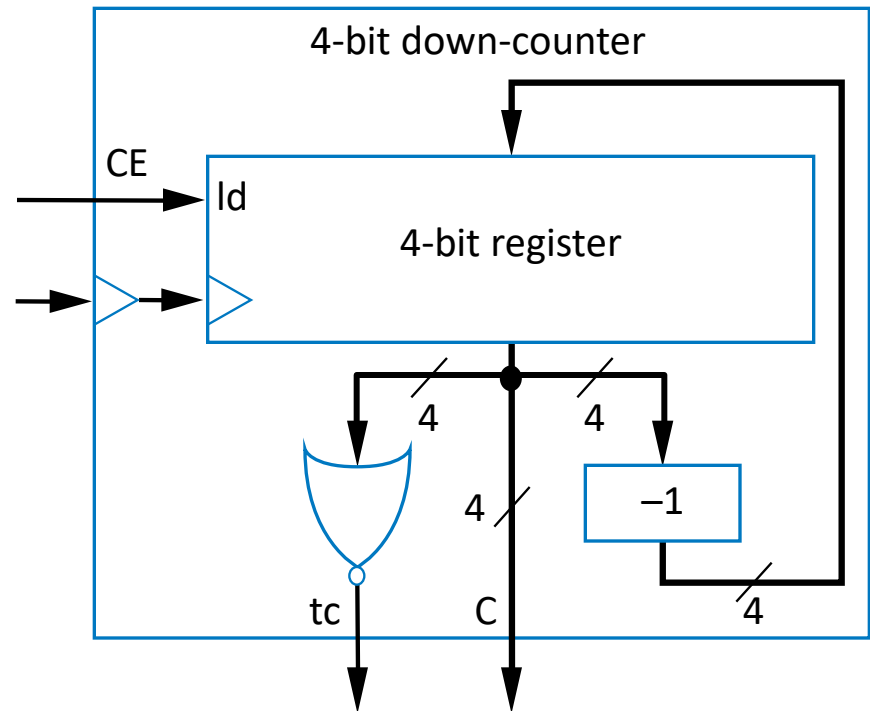
Alternative Design of Binary Counter

- Higher level (Register Transfer Level – RTL) design with combinational and sequential building blocks
 - Register
 - Incrementer
 - N-input AND gate to detect terminal count (TC), TC=1 when terminal number is reached
- Example: up counter



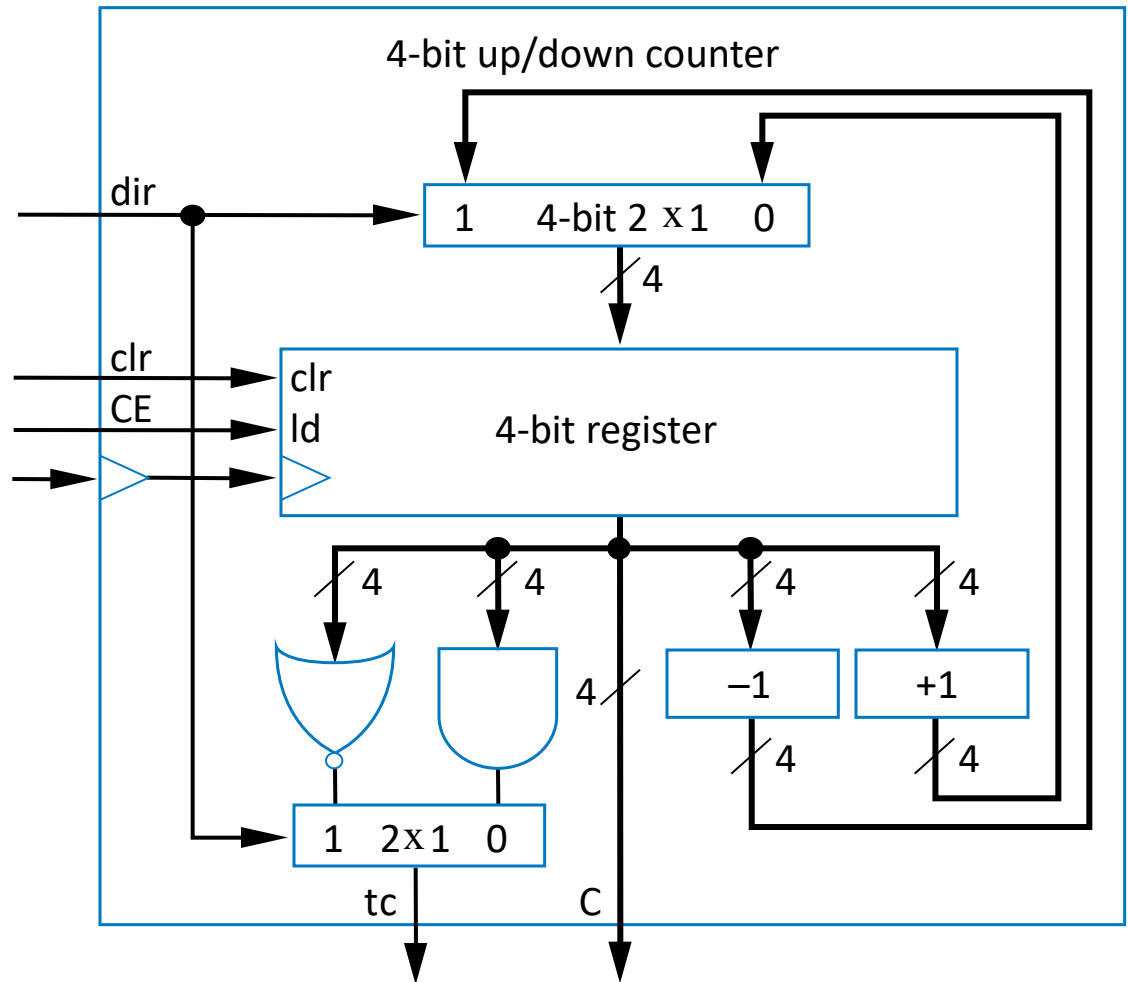
Down-Counter

- 4-bit down-counter
 - Terminal count is 0000
 - Use NOR gate for tc
 - Need decrementer (-1)



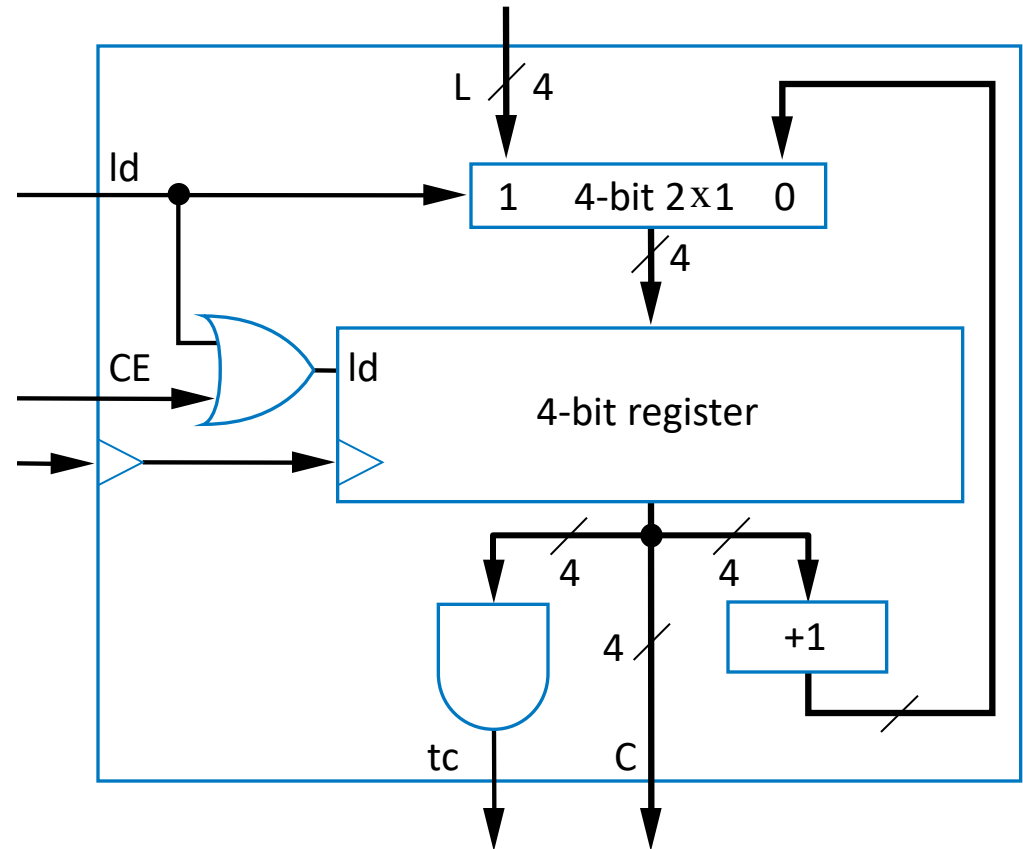
Up/Down-Counter

- Can count either up or down
 - Includes both incrementer and decrementer
 - Use dir input to select, using 2x1 MUX, dir=0 means count up
 - Likewise, dir selects appropriate terminal count value



Alternative Design of Counter with Control

- Up-counter that can be loaded with external value
 - Designed using 2x1 mux
 - Id input selects incremented value or external value
 - Load the internal register when loading external value or when counting



Summary

- What essentially does a counter do?
- How to design a synchronous counter w/ FFs and gates?
- Verilog modeling of counters
- How to integrate control signals into a counter (w/ MUX)?
- Test bench, test plan, and UUT
- Clock divider as an important application of counter
- What is clock skew? How to reduce it? (w/ synchronizer)
- How to put together a counter with building blocks?